

RELATÓRIO — PRÁTICA 2: TESTES DE CARGA COM LOCUST

Disciplina: Sistemas Distribuídos — IFPA Campus Ananindeua
Equipe: Ian Lucas Lobato Barra da Silva e Leonardo Jacomini Barcos
Alvo: <https://sd-loadlab.onrender.com>
Repositório: <https://github.com/percerval/sd-loadlab>
Infraestrutura: Render, plano Free confirmado pelo screenshot do painel; dois workers Gunicorn, quatro threads por worker. CPU/RAM efetivamente alocadas não foram verificadas no painel.
Versão do Locust: 2.46.6
Data e ambiente do gerador de carga: 01/10/2026, Debian/Linux, Python 3.13.5, ambiente virtual local .venv.

1. Objetivo

Avaliar o comportamento da API publicada sob concorrência crescente, medindo vazão (RPS), latência (mediana, p95, p99) e taxa de falhas, e investigar indícios de limites de capacidade.

2. Metodologia

- **Execução:** Locust em modo headless, a partir da máquina local, contra a URL pública HTTPS. Um health check foi realizado antes de cada cenário. Houve uma execução por cenário, sem repetições estatísticas.
- **Perfil de usuário simulado** (locustfile.py), com espera de 0,5 a 2 s entre ações:

Tarefa	Peso	Descrição
GET /items	5	listagem (I/O simulado)
GET /items/[id]	3	consulta individual
GET /	1	health check
GET /cpu?n=20000	1	20.000 iterações SHA-256

- **Cenários:** metas de 10, 50 e 150 usuários, com spawn rates de 2, 5 e 10 usuários/s, respectivamente. Cada execução dura 2 minutos totais, incluindo a rampa de aproximadamente 5, 10 e 15 segundos. As estatísticas agregadas incluem essa rampa.
- **Modelo:** cada usuário espera a resposta e depois pausa de 0,5 a 2 s; o número de usuários não equivale a requisições por segundo. /items/[id] agrupa consultas a IDs aleatórios de 1 a 50. A API utiliza dois workers e quatro threads por worker.
- **Evidências:** resultados/r_10u*, resultados/r_50u* e resultados/r_150u* contêm CSVs, histórico, HTML e logs. As execuções ocorreram aproximadamente entre 23:18–23:20, 23:23–23:25 e 23:26–23:28 UTC, respectivamente.

3. Resultados

Fonte: linha **Aggregated** dos arquivos r_XXu_stats.csv. Percentual de falhas calculado como $100 \times \text{Failure Count} / \text{Request Count}$. Percentis do Locust são aproximados; latências incluem rede e processamento.

Usuários	Requisições	Falhas (%)	RPS	Mediana (ms)	p95 (ms)	p99 (ms)
10	740	0	6,20	320	490	780
50	3.685	0	30,83	290	530	750
150	6.061	0	50,66	1.500	2.800	3.400

Os CSVs são snapshots periódicos. Os resumos finais do terminal contabilizaram 744, 3.690 e 6.067 requisições, respectivamente. Para evitar misturar instantes de coleta, a tabela e as comparações usam exclusivamente os CSVs. Os três processos terminaram com código 0, e o health check após cada teste retornou HTTP 200.
A Figura 1 reúne os três gráficos do cenário de 150 usuários. Os percentis no gráfico são temporais e podem diferir dos percentis agregados da tabela.

4. Análise

Percentis de latência. Com 150 usuários, aproximadamente 95% das requisições responderam em até 2.800 ms e 99%, em até 3.400 ms. Esses percentis evidenciam a cauda da distribuição e complementam a mediana de 1.500 ms, que sozinha não descreve as respostas mais lentas.

Capacidade e degradação. De 10 para 50 usuários, a vazão cresceu aproximadamente cinco vezes, com latências agregadas próximas. De 50 para 150, a concorrência triplicou, mas a vazão cresceu somente 1,64 vez; o p95 aumentou cerca de 5,3 vezes. Isso é compatível com aproximação de um limite de capacidade, sem demonstrar um teto exato. A rota `/cpu` apresentou o maior p95 entre as rotas no cenário de 150 usuários: 3.000 ms, contra 2.800 ms em `/items` e `/`, e 2.700 ms em `/items/[id]`.

Limitações. Não foram coletadas séries de CPU/RAM do servidor nem métricas suficientes do gerador para isolar o gargalo. Rede, filas, recursos compartilhados e capacidade do cliente podem influenciar os resultados. O health check prévio reduz a influência do despertar inicial, mas não foi realizado experimento específico de cold start. A única execução por cenário não permite atribuir significado estatístico às pequenas diferenças entre 10 e 50 usuários. Zero falhas registradas não significa baixa latência nem validação funcional de todos os corpos das respostas pelo Locust.

5. Possíveis melhorias

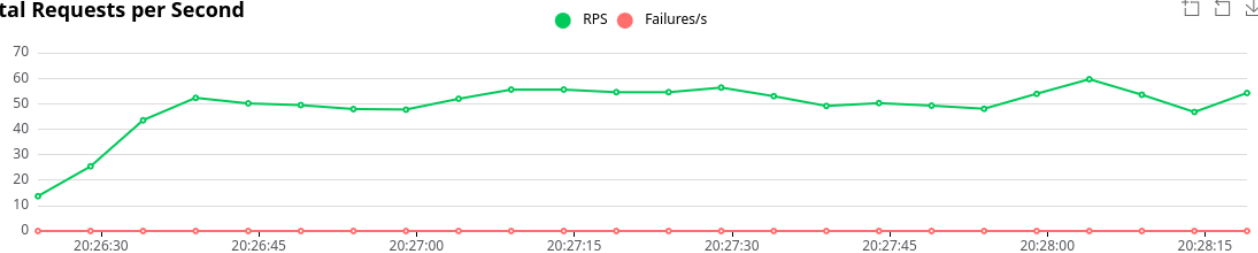
- Coletar métricas de CPU/RAM e do gerador, repetir cenários e separar testes de I/O e CPU para identificar o gargalo.
- Avaliar workers/threads e escala vertical conforme os recursos e o gargalo medido; mais workers não garantem maior vazão.
- Avaliar múltiplas réplicas com balanceamento se a infraestrutura permitir; a API stateless facilita essa estratégia. Essas melhorias não foram implementadas ou medidas nesta prática.

6. Conclusão

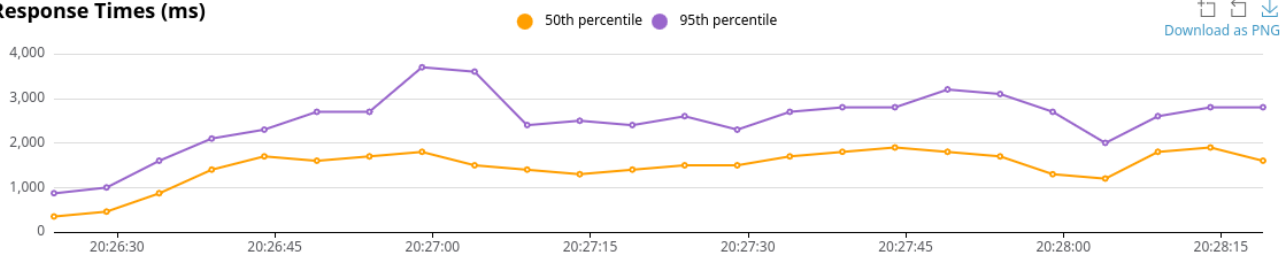
Nos cenários de 10 e 50 usuários, a vazão cresceu aproximadamente na proporção da concorrência, sem grande alteração de latência. Com 150 usuários, houve degradação acentuada, com p95 de 2,8 s e crescimento proporcionalmente menor da vazão, apesar de nenhuma falha registrada. Os resultados mostram que disponibilidade não garante boa experiência de uso e sugerem restrição de capacidade sob maior carga. Sem métricas adicionais, não é possível afirmar que a CPU saturou ou determinar o limite físico exato da instância.

Evidência gráfica

Total Requests per Second



Response Times (ms)



Number of Users

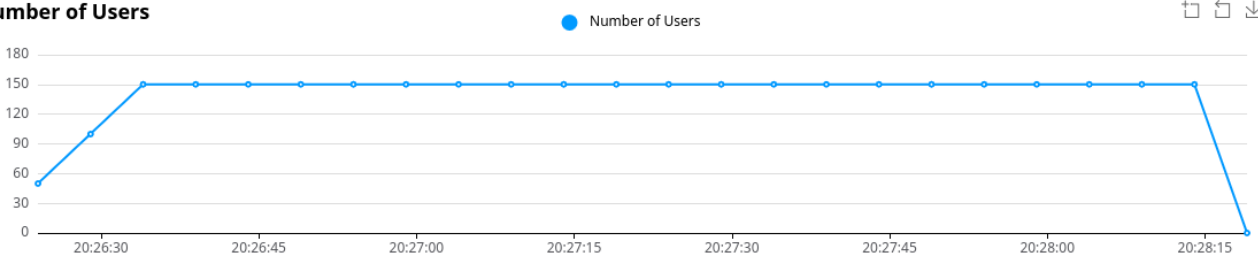


Figura 1 — Cenário de 150 usuários, rampa de 10 usuários/s e duração total de 2 minutos: requisições e falhas por segundo (topo), latências p50/p95 (centro) e usuários ativos (base). Horários exibidos no fuso local UTC-3. A queda final para zero usuários corresponde ao encerramento do teste, não à queda do serviço. Fonte: relatório HTML do Locust da execução `r_150u`.